

Quantas vezes mais rápido o Numpy foi, no seu teste?

O Numpy foi 53.4x mais rápido

Isso é SIMD ou MIMD? Justifique usando a definição de cada um.

É um SIMD pois apenas solicitamos a questão em 'soma', se fosse um MIMD teria que ter mais questões com operadores diferentes para vários outros dados.

Compare esse resultado com o da corrida CPU x GPU da Aula 1 (Exercício 3).

Por que a diferença de velocidade ali foi tão maior do que aqui?

Pois a SIMD executa apenas uma tarefa sozinho, já a gpu (maioria dos casos) são vários executores realizando uma tarefa (o que torna o trabalho mais rápido e prático)

Intel Core i9 (14ª geração) — RISC ou CISC?

Os dois, porque pode ter tanto instruções complexas quanto instruções simples.

Apple M3 — RISC ou CISC?

CISC pois realiza instruções pouco complexas

ARM Cortex-A78 (usado em boa parte dos celulares Android) — RISC ou CISC?

RISC, porque é mais fácil de fazer o pipeline (buscar, decodificar, executar, memorizar e escrever)

AMD Ryzen 9 — RISC ou CISC?

RISC, instruções simples para mais facilidade em realizar pipeline.

Um núcleo CUDA de uma GPU NVIDIA — se aproxima mais de qual filosofia, e por quê?

é 100% focado em operações simples e diretas, como somas, multiplicações e operações lógicas em um único ciclo.

Bônus: as CPUs Intel/AMD modernas são CISC 'por fora', mas por dentro traduzem cada instrução CISC em microinstruções parecidas com RISC antes

de executar. Pesquise esse conceito (chamado de 'micro-ops') e explique, em 2-3 frases, por que os fabricantes fazem isso.

Os fabricantes fazem isso para unir compatibilidade com alto desempenho. Com isso a CPU consegue executar internamente operações simples e padronizadas no estilo RISC (instruções muito simples e mais fácil de fazer pipeline. Dividir instruções complexas em etapas simples permitindo que a CPU trabalhe em “esteira” (sem pausas), antecipando decisões futuras e executando várias tarefas ao mesmo tempo sem travar.

Quantos processos apareceram na sua saída do ps aux?

14

Ache pelo menos um processo com STAT igual a S (Sleeping) e um com R (Running), se houver. O que você acha que o processo em S está esperando? Não tem nada.

Compare a coluna PID daqui com o PID que aparecia no nvidia-smi da Aula 1 — são o mesmo tipo de número (identificador de processo)? O processo Python do seu notebook aparece nas duas listas?

Sim. sim, aparece.

Sem pipeline (uma instrução só começa depois que a anterior termina TUDO): quantos ciclos levam 4 instruções? E 10 instruções? (fórmula: $N \text{ instruções} \times 5 \text{ ciclos}$)

Não entendi

Com pipeline (como no diagrama que vimos): quantos ciclos levam 4 instruções? E 10 instruções? (fórmula: 5 ciclos pra 'encher' o pipeline + 1 ciclo por instrução extra)

Não entendi

Calcule o speedup (quantas vezes mais rápido) do pipeline pra 4 instruções, e de novo pra 10 instruções. O speedup aumenta, diminui, ou fica igual conforme o número de instruções cresce?

Não entendi

Bônus: se o número de instruções crescesse até o infinito, pra que valor o speedup se aproximaria? (Dica: pense no que acontece com o '+5 ciclos iniciais' quando N é gigante.)

Pesquise e liste pelo menos 2 vantagens reais que a Apple (ou analistas de mercado) apontaram nessa migração — cite as fontes.

Maior Eficiência Energética (*Performance por Watt*) e Bateria Significativamente Superior : Os Macs passaram a entregar um desempenho comparável ou superior aos chips de computador de mesa, mas consumindo uma fração da energia. Na prática, isso resultou em MacBooks com o dobro de duração de bateria e que esquentavam tão pouco que o MacBook Air pôde eliminar a ventoinha interna. (**Declaração Oficial da Apple (WWDC 2020/Release)**: A Apple justificou a mudança afirmando que o objetivo era alcançar “*desempenho líder da indústria por watt*”).

Escreva um parágrafo (5-8 frases) conectando essas vantagens com os conceitos de RISC vistos hoje: simplicidade de instrução, facilidade de pipeline, e o que isso significa pra consumo de energia (importante pra bateria de notebook).

A transição para o Apple Silicon mostra a força da arquitetura RISC na prática. Por usar instruções simples e de tamanho fixo, o processador precisa de uma unidade de decodificação menor e menos complexa. Isso é mais fácil para o pipeline, permitindo processar várias instruções por ciclo de forma fluida e sem gargalos. Como o chip

gasta menos transistores com controle complexo, ele gera menos calor e consome muito menos eletricidade. O resultado direto para os notebooks é um desempenho altíssimo com uma autonomia de bateria sem precedentes.

Rode o código pelo menos 3 vezes. O contador final deu igual nas 3 vezes? Deu igual ao valor esperado (400000) em alguma delas? Sim

Com suas palavras: porque threads que compartilham a mesma variável podem produzir um resultado 'errado', mesmo que cada uma, sozinha, esteja fazendo a conta certa?

Porque ocorre a condição corrida (ordem e o tempo de leitura e escrita se entrelaçaram no lugar errado). Sendo o resultado de quem chega primeiro na memória.

Isso aconteceria da mesma forma se, em vez de 4 threads do mesmo processo, fossem 4 processos separados, cada um com sua própria cópia da variável contador? Justifique usando a diferença entre processo e thread vista hoje.

Não entendi muito bem